

Project Report

on

***FINOPS –“Automated Intelligent Document Processing system for
Financial operation(Amount Payable)”***

By

YOURAJ RANA

Under the Guidance of

ANIL PHALKE

Master of Computer Application



**P .E. Society's
Modern Institute of Business Studies (Autonomous)
Nigdi, Pune**

2025-2026



Progressive Education Society's
Modern Institute of Business Studies (Autonomous)
Nigdi, Pune-411044

Certificate

This is to certify that the **Online Job Training (OJT-2441)** entitled ***FINOPS “Automated Intelligent Document Processing system for Financial operation(Amount Payable)”*** is submitted by **YOURAJ RANA** a bonafide student of this institute, studying for a Master of Computer Application, has successfully carried out MCA semester-IV. This project report is submitted in partial fulfilment of the **M.C.A. Semester IV Academic Year 2025-26** curriculum as per the Institute norms.

Dr. Maithili Arjunwadkar
Director
PES MIBS

Project Guide

External Examiner

DECLARATION

I, the undersigned, solemnly declare that the report of the project work **FINOPS** is based on my own work carried out during the course of my study under the supervision of **ANIL PHALKE**. I assert that the statements made and conclusions drawn are an outcome of the project work.

I further declare that to the best of my knowledge and belief, the project report does not contain any part of any work which has been submitted for the award of any other degree/diploma/certificate in this Institute or University or any other University.

YOURAJ RANA

Date:

Place:

ACKNOWLEDGEMENT

It is my proud privilege to express my profound gratitude to the entire management of Progressive Education Society's Modern Institute of Business Studies (Autonomous) and Faculty members of the institute for providing me with the opportunity to avail the excellent facilities and infrastructure of the institute. The knowledge and values inculcated have proved to be of immense help to my career.

I am grateful to **Prof. Dr. Maithili Arjunwadkar (Director, MIBS)** for their astute guidance, constant encouragement and sincere support for this project work.

I express my sincere gratitude to **ANIL PHALKE** for their inspiration, constructive suggestion, mastermind analysis and affectionate guidance in my work, without which this project work completion would have been impossible for me. Sincere thanks to all my seniors and colleagues at the company for their support and assistance throughout the project.

I would like to thank all the faculty for providing me with an opportunity to pursue my industrial training, as it is an important part of the MCA course.

INDEX

Chapter	Content	Page Number
Chapter 1	Introduction	7-11
1.1	Company Profile/Institute Profile/Client Profile	7
1.2	Abstract	8
1.3	Existing System and Need for System	9
1.4	Scope of System	10
1.5	Operating Environment-Hardware and Software	10
1.6	Brief Description of Technology Used Operating Systems used (Windows or Unix) Databases	11
Chapter 2	Proposed System	12-15
2.1	Study of Similar Systems	12
2.2	Feasibility Study	13
2.3	Objectives of Proposed System	14
2.4	Users of System	15
Chapter 3	Analysis and Design	16-25
3.1	System Requirements (Functional and Non-Functional requirements)	16-17
3.2	Entity Relationship Diagram (ERD)(if applicable)	18
3.3	Use Case Diagrams	19
3.4	Class Diagram	20
3.5	Activity Diagram	21
3.6	Table Structure	22-23
3.7	Data Dictionary	23
3.8	Sample Input and Output Screens	24-28
Chapter 4	Coding	26 -30
4.1	Algorithm	27
4.2	Code Snippets	28

Chapter 5	Testing	29- 31
5.1	Test Strategy	29
5.2	Unit Test Plan	29
5.3	Acceptance Test Plan	29
5.4	TestCase/Test Script	31
Chapter 6	Limitations of the Proposed System	32
Chapter 7	Proposed Enhancement	33
Chapter 8	Conclusion	34
Chapter 9	Bibliography	34-35
Chapter 10	Publication/Competition Certificates	
	Annexure -Progress Sheet	

Chapter 1: Introduction

1.1 Company Profile

Konpo Pvt. Ltd. is a privately held information technology services company headquartered in Pune, Maharashtra, India. The company specialises in the design, development, and deployment of full-stack web applications and enterprise-grade software solutions tailored to the needs of modern businesses. With a highly skilled team of software engineers, UI/UX designers, and project managers, Konpo Pvt. Ltd. delivers end-to-end digital products across a range of domains including e-commerce, education technology, healthcare, and enterprise resource planning.

The organization follows agile development methodologies, enabling rapid iteration and high-quality deliverables within defined timelines. Konpo Pvt. Ltd. places a strong emphasis on adopting modern technology stacks — including the MERN stack (MongoDB, Express.js, React.js, Node.js) — to build scalable, maintainable, and performant applications. As part of its commitment to knowledge transfer and workforce development, the company actively engages students through On-the-Job Training (OJT) programmes, providing hands-on exposure to real-world software development practices.

The present project — a Web-Based EdTech Application — was conceived, designed, and developed during the course of the OJT at Konpo Pvt. Ltd. under the direct supervision of the company's technical mentors.

1.2 Abstract

In today's corporate world, the Accounts Payable department wastes countless hours manually reading invoices, typing data, and checking for errors. This traditional approach is not only incredibly slow but also leads to expensive mistakes—like accidentally paying the same bill twice or reimbursing restricted items .

To solve this, my project introduces an **AI-Powered Financial Operations Automation System**. Think of it as an intelligent digital assistant that automates the entire invoice pipeline.

First, the system acts as a security guard. It uses cryptographic hashing (MD5) to check the digital fingerprint of every uploaded file, instantly blocking duplicate invoices so a company never pays the same bill twice. Once verified, it uses advanced AI (Google Gemini Vision/OCR 2.0) to intelligently read unstructured PDFs and images, extracting key details like vendor names, subtotals, and item lists.

But it doesn't just read the data—it audits it. The system automatically recalculates the math to catch vendor errors and scans the item list for company policy violations. If a discrepancy is found, our automated **Dispute Agent** drafts and sends an email directly to the vendor to resolve the issue, completely removing the human operator from the tedious back-and-forth.

Everything is secured with a Role-Based Access Control (RBAC) system, ensuring that Operators handle the daily processing while Admins have access to a real-time financial analytics dashboard and secure audit logs. Ultimately, this project proves how AI can transform a frustrating, error-prone chore into a fast, secure, and smart automated workflow.

1.3 Existing System and Need for System

Before we built this software, we had to look at how companies currently handle their bills.

The Existing System (How it works right now): Currently, most mid-sized companies rely on a highly manual "Maker-Checker" system.

1. **Manual Data Entry:** An operator receives an invoice (either paper or a PDF email attachment) and manually types the vendor name, date, and amounts into an Excel sheet or basic accounting software like Tally.
2. **Human Auditing:** A manager then manually reads the invoice to check if the math adds up ($\text{Subtotal} + \text{Tax} = \text{Total}$) and scans the items to ensure no restricted goods were purchased.
3. **Manual Dispute:** If the manager finds an error, they have to open their email, find the vendor's contact info, type out a complaint, and wait for a reply.

The Need for the Proposed System (Why we built this): The existing manual system is failing for several reasons:

- **High Rate of Human Error:** When an operator types 100 invoices a day, they will eventually make a typo or miss a calculation error, costing the company money.
- **The "Duplicate Payment" Trap:** If a vendor accidentally emails the same invoice twice (a week apart), a human operator might not remember paying it and will process it again. Manual systems cannot catch digital duplicates easily.
- **Wasted Time on Disputes:** Writing emails to vendors about a \$5 math mistake takes up valuable time that finance teams could spend on actual financial planning.
- **Lack of Real-Time Analytics:** In manual systems, management has to wait until the end of the month for an operator to generate an Excel report to see total spending.

1.4 Scope of System

The scope defines the exact boundaries of our project—what it is built to do, and just as importantly, what it is *not* built to do. For this project, we focused on automating the most painful parts of the Accounts Payable workflow.

In-Scope (What the system handles):

- **Intelligent Document Processing:** The system accepts invoice uploads (PDFs and images) and uses AI to extract key fields (Vendor Name, Total Amount, Item Lists) without manual data entry.
- **Duplicate Fraud Prevention:** Every uploaded file is instantly scanned using an MD5 cryptographic hash to ensure the exact same file hasn't been processed before.
- **Automated Auditing & Validation:** The system mathematically checks if the subtotal plus tax equals the grand total. It also scans item lists for company policy violations (e.g., restricted items like alcohol).
- **Automated Dispute Resolution:** If an invoice fails the audit, the system automatically drafts and sends an email to the vendor explaining the error, waiting for a corrected invoice.

- **Role-Based Dashboards:** The system supports a multi-user environment. 'Operators' handle daily uploads, while 'Admins' view high-level analytics and audit logs.
- **Data Export:** Admins can download the processed financial ledgers into an Excel/CSV format for external auditing.

1.5 Operating Environment - Hardware and Software

Since this is a web-based application powered by Cloud AI, it does not require a supercomputer to run. However, the machine hosting the application needs reliable internet access.

Hardware Requirements (Server / Host Machine):

- **Processor:** Intel Core i3 / AMD Ryzen 3 (Minimum) | Intel Core i5 or higher (Recommended for faster data processing).
- **RAM:** 4 GB (Minimum) | 8 GB (Recommended to handle Pandas DataFrames and Streamlit UI smoothly).
- **Storage:** 20 GB of free space (For the operating system, MySQL database, and storing uploaded invoice files).
- **Network:** An active, stable internet connection is **mandatory**. The system must connect to the Google Cloud servers for the Gemini API and to the Gmail SMTP servers to send emails.

Software Requirements:

- **Operating System:** Windows 10/11 or any standard Linux distribution (e.g., Ubuntu). The system is platform-independent.
- **Programming Language:** Python 3.9 or higher.
- **Database Management:** MySQL Server (can be run via XAMPP, WAMP, or standalone MySQL Workbench).
- **Client Interface:** Any modern web browser (Google Chrome, Microsoft Edge).

1.6 Brief Description of Technology Used

To build this pipeline, we selected a modern, lightweight, and highly efficient technology stack.

- **Python (Core Engine):** Python serves as the backbone of our business logic. We chose Python because of its massive ecosystem of data processing libraries and its native support for AI API integrations.
- **Streamlit (Frontend / Web Framework):** Instead of writing complex HTML, CSS, and JavaScript, we used Streamlit. It is an open-source Python framework designed specifically for machine learning and data science projects. It allowed us to build interactive web dashboards and upload interfaces rapidly.
- **Google Gemini Vision API (OCR 2.0 / AI Extraction):** This is the "brain" of our extraction module. Traditional OCR (Optical Character Recognition) just reads text left-to-right and gets confused by complex table layouts. Gemini is a *Multimodal Generative AI*, meaning it actually "understands" the structure of the invoice, reliably pulling the exact Vendor Name and Totals regardless of how the invoice is formatted.
- **MySQL (Database):** We used a relational database to store our structured data. It handles the users, invoices, and audit_logs tables, ensuring that all our financial data obeys the principles of ACID (Atomicity, Consistency, Isolation, Durability).
- **Cryptography (hashlib):** We used Python's built-in hashing library to generate the 32-character MD5 fingerprints for our duplicate detection system.
- **SMTP (smtplib):** Simple Mail Transfer Protocol is used to programmatically connect to Gmail's servers and automate the dispatch of our dispute emails to vendors.
- **Pandas & ExcelWriter:** Used in the analytics module to convert our MySQL database records into clean, downloadable Excel reports for the Admin.

Chapter 2: Proposed System

2.1 Study of Similar Systems

1. **Manual ERP Systems (e.g., basic Tally or legacy SAP):** In these systems, operators look at a physical invoice and manually type the details into the software. It is incredibly slow, highly prone to human error (typos), and offers zero automated validation. If an operator types \$500 instead of \$50, the system blindly accepts it.
2. **Traditional OCR Tools (e.g., ABBYY FineReader):** These tools scan documents and read text. Traditional OCR relies on strict templates. If a vendor moves the "Total Amount" from the bottom right to the top left of their invoice, the OCR fails and extracts the wrong data. It cannot "understand" the document.
3. **Enterprise Expense Management (e.g., SAP Concur):** These are massive, cloud-based tools that handle expenses well. They are prohibitively expensive for small-to-medium enterprises (SMEs). Furthermore, configuring custom rules (like our specific "alcohol" policy flag) requires expensive consultants.

2.2 Feasibility Study

- **Technical Feasibility .** The technology stack chosen is highly robust and accessible. Python provides excellent libraries for data manipulation (Pandas) and web development (Streamlit). Google's Gemini API handles the heavy lifting of AI image processing, meaning we don't need to train a massive machine learning model from scratch. MySQL easily handles the relational data for our users and audit logs. The hardware requirements are minimal because the AI processing happens in the cloud.
- **Operational Feasibility :** The system was designed with a "Zero-Training" philosophy. The user interface mimics the familiar act of uploading an email attachment. Furthermore, it perfectly maps to the existing "Maker-Checker" corporate workflow. Operators do the data entry (uploading), and Admins do the checking (analytics). It solves a massive headache for the Accounts Payable team without forcing them to learn a complicated new tool.

- **Economic Feasibility :** This project is highly economically viable. Instead of paying thousands of dollars for enterprise software licenses, this system relies heavily on open-source technologies (Python, Streamlit, MySQL). The only variable cost is the API usage for Google Gemini, which is a pay-as-you-go model (costing fractions of a penny per invoice). The Return on Investment (ROI) is massive when you consider the money saved by blocking duplicate payments and catching vendor calculation errors.

2.3 Objectives of Proposed System

The primary goal of this project is to automate the Accounts Payable pipeline. Specifically, the system is engineered to achieve the following objectives:

1. **Automate Data Extraction:** To eliminate manual data entry by using Multimodal AI to instantly read and parse vendor names, dates, subtotals, taxes, and itemized lists from unstructured PDFs and images.
2. **Prevent Duplicate Payments:** To implement a cryptographic hashing mechanism (MD5) that acts as an impenetrable gatekeeper, ensuring no file is ever processed or paid twice.
3. **Enforce Mathematical & Policy Audits:** To programmatically verify the mathematical accuracy of every invoice and cross-reference purchased items against restricted company policies (e.g., flagging alcohol or personal expenses).
4. **Automate Vendor Disputes:** To drastically reduce communication delays by automatically drafting and sending contextual emails to vendors whenever an anomaly or error is detected.
5. **Provide Actionable Intelligence:** To equip management with a real-time analytics dashboard that visualizes spending trends and allows for seamless export of financial ledgers to standard formats like Excel.
6. **Ensure Strict Security & Compliance:** To maintain a completely transparent and immutable Audit Log that tracks exactly "who did what and when," ensuring accountability for every action taken within the system.

2.4 Users of System

To maintain security and operational efficiency, the system implements a Role-Based Access Control (RBAC) architecture. There are two distinct types of users who will interact with the system:

1. The Operator (The "Maker")

- **Profile:** Data Entry Clerks, Junior Accountants, or Interns.
- **Role:** They are the operational workforce. Their primary job is to feed invoices into the system and handle immediate task-level decisions.
- **Access:** They have access to the "Operations Center" where they upload files, review the AI's extraction results, and trigger automated dispute emails if an error is flagged. For security reasons, they *cannot* see company-wide analytics, total spending, or the system audit logs.

2. The Admin (Management)

- **Profile:** Finance Managers.
- **Role:** They provide oversight and make strategic decisions based on the aggregated data.
- **Access:** Admins have access to the "Analytics Dashboard" to view company-wide financial charts. They can download the complete monthly database to Excel for tax filing. Crucially, they have exclusive access to the "Security Audit Trail," allowing them to monitor the Operators' actions and ensure no fraudulent deletions or unauthorized activities are taking place.

Chapter 3: Analysis and Design

3.1 System Requirements (Functional and Non-Functional Requirements)

A. Functional Requirements (FRs)

- **FR-01: User Authentication & Role Management:** The system must provide a secure login mechanism. It must differentiate between 'Operators' and 'Admins' based on database records, dynamically altering the user interface and module access based on the logged-in user's role.
- **FR-02: Cryptographic Duplicate Detection:** Upon file upload, the system must instantly generate an MD5 hash of the file. It must query the invoices database to check if this hash already exists. If a match is found, the system must block the upload and alert the user to prevent duplicate processing.
- **FR-03: AI-Powered Data Extraction:** The system must accept unstructured PDF documents and images and transmit them securely to the Google Gemini Vision API. It must successfully parse and return specific key-value pairs: Vendor Name, Vendor Email, Invoice Date, Subtotal, Tax, and Grand Total.
- **FR-04: Automated Mathematical Audit:** The system must programmatically verify the vendor's math. It must calculate Subtotal + Tax and compare it against the extracted Grand Total. If there is a mismatch, the invoice status must be automatically flagged as "Math Error."
- **FR-05: Policy Compliance Scanning:** The system must scan the extracted line items against a predefined list of restricted company keywords . If a match is found, it must flag the invoice as a "Policy Violation."
- **FR-06: Automated Dispute Resolution (Email Agent):** For any flagged invoice, the system must display a "Draft Dispute Email" button to the Operator. Upon clicking, the system must automatically construct a contextual email (using smtplib) detailing the exact error and send it directly to the vendor's extracted email address.
- **FR-07: Management Analytics Dashboard:** The system must provide an Admin-only dashboard that visualizes processed data (e.g., Total Spend, Flagged vs. Approved ratios) using interactive charts.

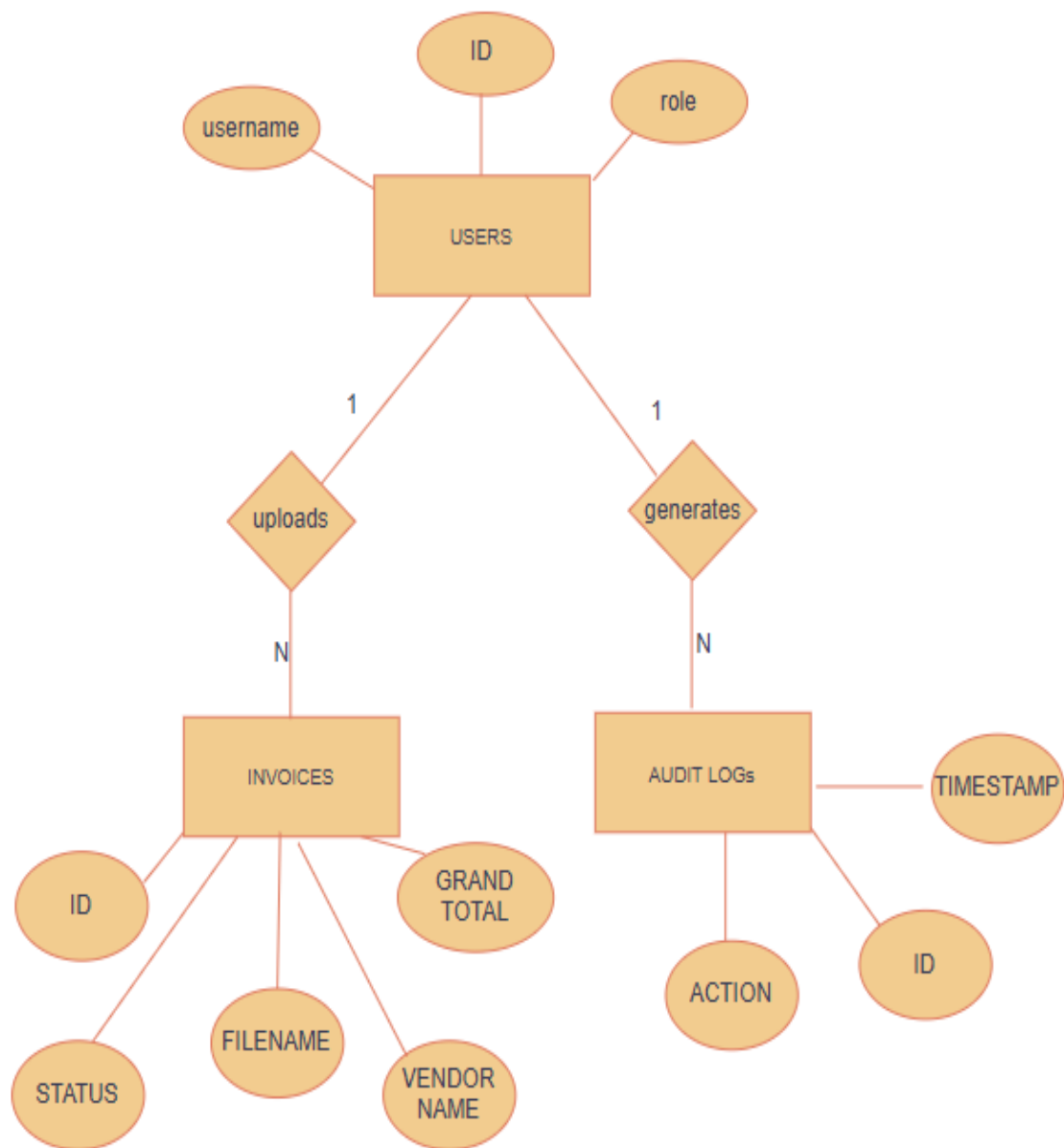
- **FR-08: Data Export to Excel:** The system must allow Admins to download the complete ledger of processed invoices into a standard .xlsx or .csv format for external accounting purposes.
- **FR-09: Immutable Audit Logging:** The system must silently record every critical action (Login, Upload, Delete, Email Sent) along with the Timestamp and User ID in an audit_logs table to ensure complete forensic traceability.

B. Non-Functional Requirements (NFRs)

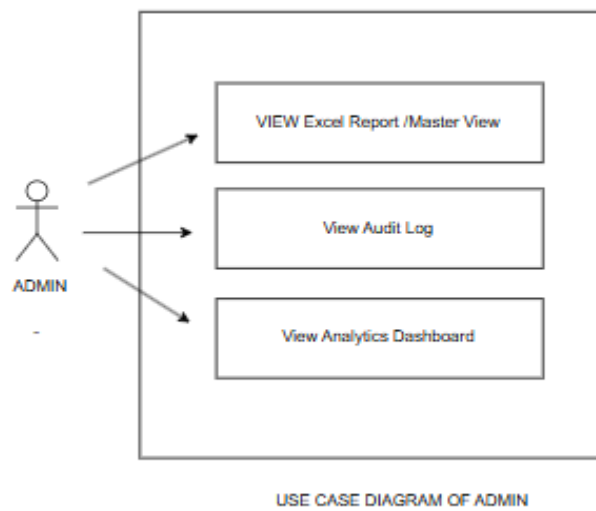
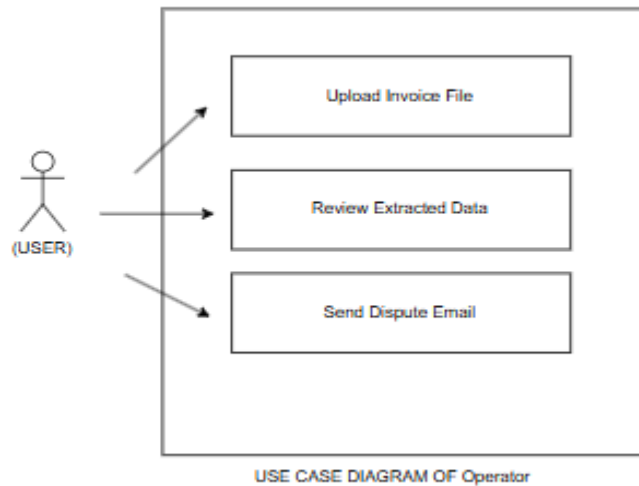
Non-functional requirements describe the quality, performance, and security constraints of the system.

- **Performance & Speed:** The system should process an uploaded invoice (Hash Check AI Extraction Math Audit) in under 10 seconds, assuming a stable internet connection to the Gemini API.
- **Security & Privacy:** Passwords stored in the database must not be plain text; they must be hashed using SHA-256 encryption. Furthermore, the system must enforce the Principle of Least Privilege, completely hiding analytics and logs from junior Operators.
- **Usability (Zero-Training UI):** The interface must be highly intuitive, utilizing a standard drag-and-drop file uploader and clear color-coded statuses (Green for Approved, Red for Flagged) so that non-technical accounting staff can use it without formal training.
- **Reliability & Exception Handling:** If the Gemini API is temporarily down or an email fails to send due to a bad address, the system must not crash. It must gracefully catch the exception, display a user-friendly error message, and update the database status to "Manual Intervention Required."

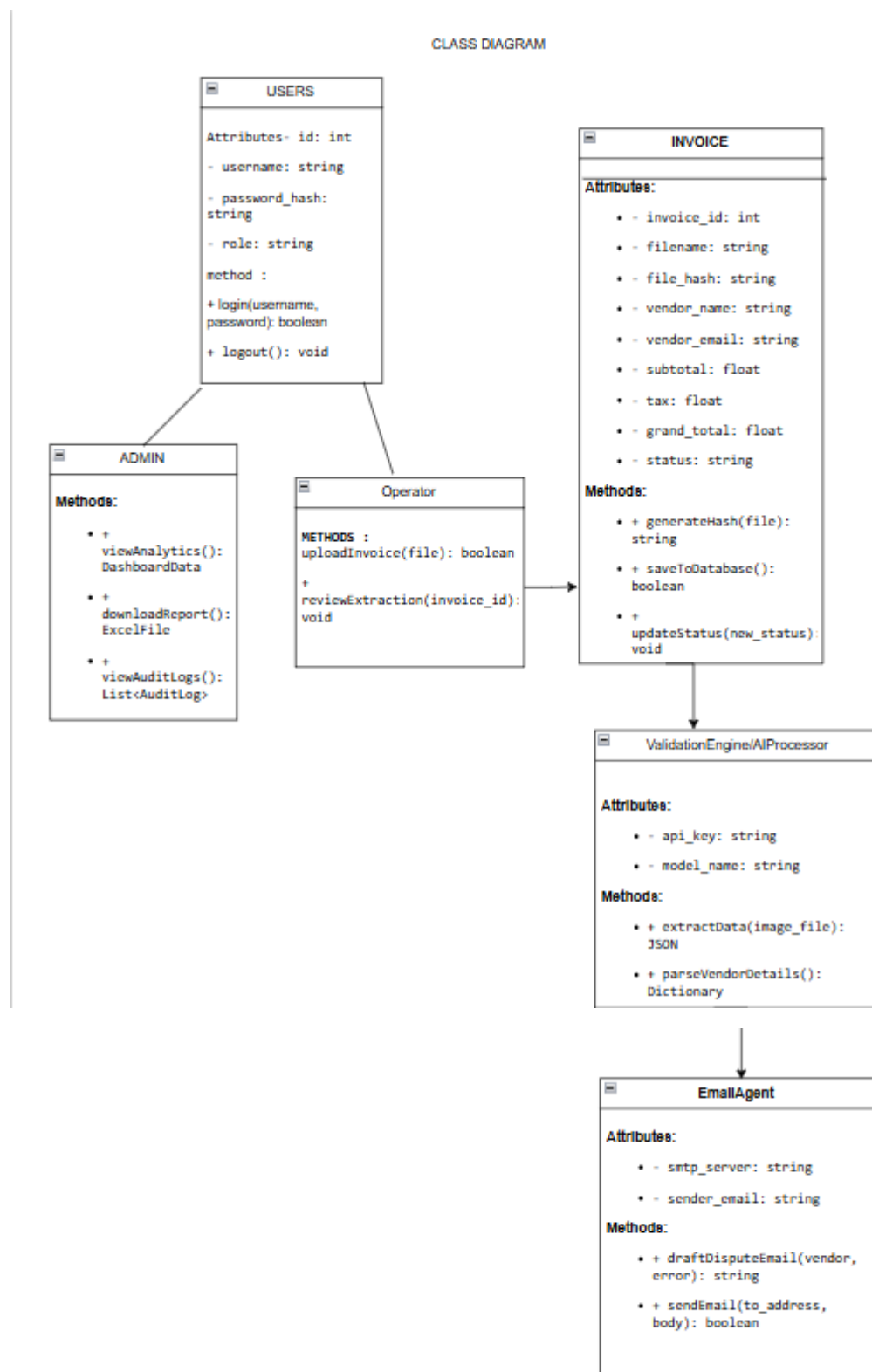
3.2 Entity Relationship Diagram



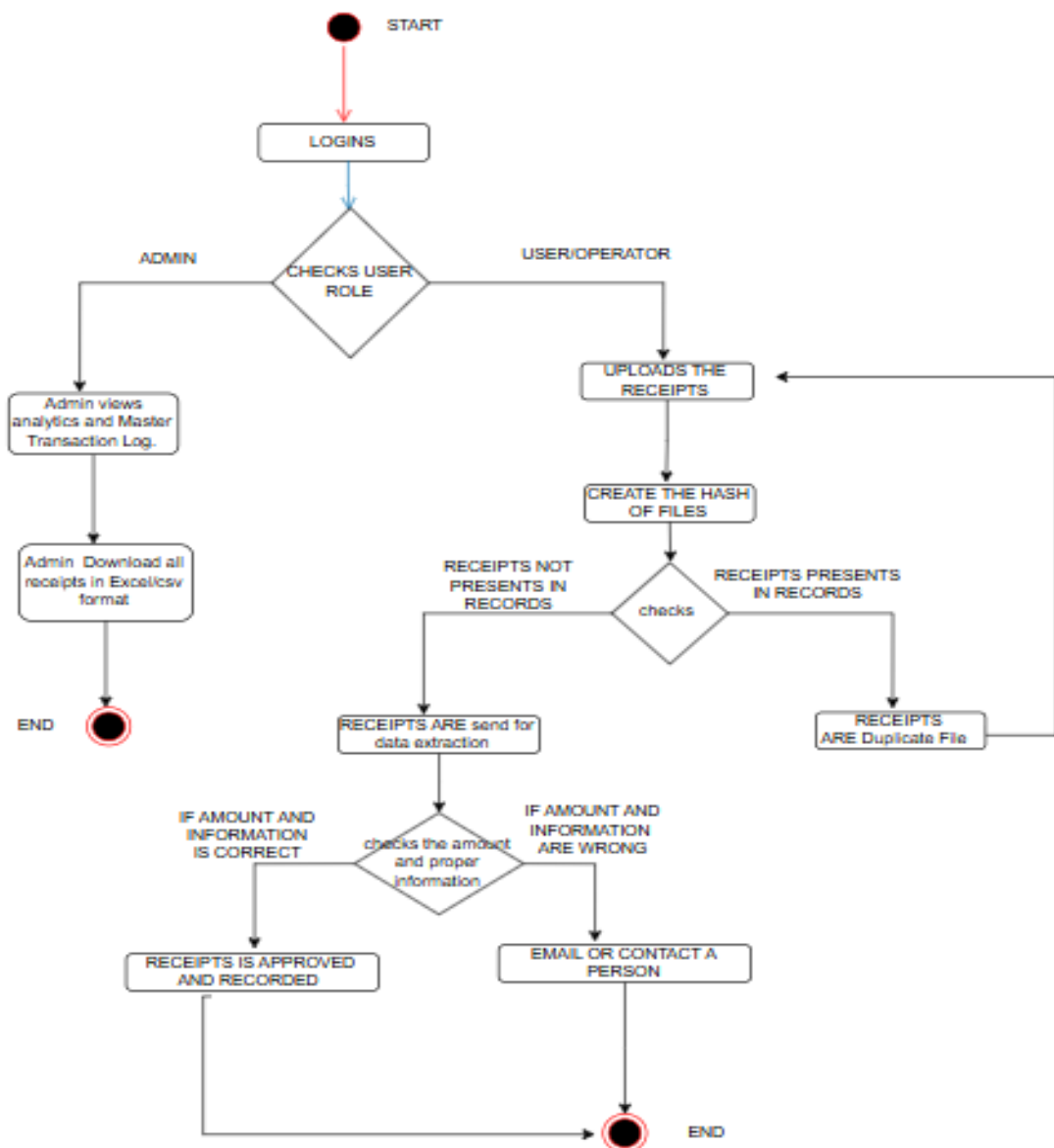
3.3 Use Case Diagrams



3.4 Class Diagram:



3.5 Activity Diagram



3.6 Table Structure

The database architecture for the FinOps Automation System is designed using MySQL. To ensure data integrity, eliminate redundancy, and maintain strict security, the database is normalized up to the Third Normal Form (3NF). The system is built upon three foundational tables connected via Foreign Key constraints.

Table 1: Users

This table acts as the central authentication hub. It stores the credentials and access levels for every individual interacting with the system.

- **Primary Key:** id
- **Relationships:** One User can upload many Invoices (1:N). One User can trigger many Audit Logs (1:N).

Table 2: invoices

This is the master transaction table. It stores both the physical metadata of the uploaded file (like its cryptographic hash) and the financial data extracted by the Gemini AI.

- **Primary Key:** id
- **Foreign Key:** uploaded_by (References users.id)
- **Relationships:** Each invoice belongs to exactly one user who uploaded it.

Table 3: audit_logs

This table functions as the immutable security layer. It silently tracks every critical action performed in the system to ensure compliance and accountability.

- **Primary Key:** id
- **Foreign Key:** user_id (References users.id)
- **Relationships:** Each log entry is tied to the exact user who performed the action.

3.7 Data Dictionary

A Data Dictionary is a detailed description of the data structures used within the system. It defines the data type, size, constraints, and purpose of every single attribute in our database.

1. Data Dictionary for users Table

Column Name	Data Type	Size	Constraint	Description / Purpose
id	INT	11	PK , Auto Increment	Unique identifier for each system user.
username	VARCHAR	50	NOT NULL, UNIQUE	The login ID used by the Operator or Admin.
password_hash	VARCHAR	255	NOT NULL	Stores the SHA-256 encrypted password. Plain text passwords are never saved.
role	ENUM	-	DEFAULT 'User'	Defines RBAC. Allowed values are exactly: 'Admin' or 'User'.
email	VARCHAR	100	NULL	The employee's corporate email address (optional for password resets).

Column Name	Data Type	Size	Constraint	Description / Purpose
created_at	TIMESTAMP	-	DEFAULT CURRENT_TIMESTAMP	Automatically records the exact date and time the account was created.

2. Data Dictionary for invoices Table

Column Name	Data Type	Size	Constraint	Description / Purpose
id	INT	11	PK , Auto Increment	Unique internal identifier for the processed document.
filename	VARCHAR	255	NOT NULL	The original name of the uploaded PDF or image file.
file_hash	VARCHAR	32	NOT NULL, UNIQUE	The 32-character MD5 cryptographic fingerprint used to instantly block duplicate uploads.
vendor_name	VARCHAR	100	NULL	Extracted by AI: The name of the billing company.
vendor_email	VARCHAR	150	NULL	Extracted by AI: The vendor's contact email, required by the automated Dispute Agent.
subtotal	DECIMAL	10,2	NULL	Extracted by AI: The base amount before taxes. (<i>DECIMAL is used instead of FLOAT to prevent rounding errors in money</i>).
tax_amount	DECIMAL	10,2	NULL	Extracted by AI: The total tax applied to the bill.
grand_total	DECIMAL	10,2	NULL	Extracted by AI: The final amount claimed by the

Column Name	Data Type	Size	Constraint	Description / Purpose
				vendor. Used for Analytics dashboards.
status	VARCHAR	50	DEFAULT 'Pending'	Stores the outcome of the Python Validation logic (e.g., 'Approved', 'Flagged (Math Error)').
error_reason	TEXT	-	NULL	If flagged, this stores the exact reason (e.g., "100 + 10 != 120") so the Admin can review it later.
uploaded_by	INT	11	FK	Links to users.id. Proves exactly which Operator uploaded the file.
upload_time	TIMESTAMP	-	DEFAULT CURRENT_TIMESTAMP	The exact server time the file was submitted to the database.

3. Data Dictionary for audit_logs Table

Column Name	Data Type	Size	Constraint	Description / Purpose
id	INT	11	PK , Auto Increment	Unique identifier for the log entry.
user_id	INT	11	FK	Links to users.id. Identifies the actor who triggered the log.
action	VARCHAR	255	NOT NULL	A text description of what happened (e.g., "Drafted Dispute Email to Dell").
timestamp	TIMESTAMP	-	DEFAULT CURRENT_TIMESTAMP	

3.8 Sample Input and Output Screens:

<<

admin

Navigate

Manage Invoices

Security Audit

Analytics Dashboard

Logout

Deploy

Master Invoice Ledger

	filename	vendor_name	Original Amount	status	processed_by	upload_time
1	Screenshot 2026-03-28 003533.png	testing1	180	Approved	user	2026-03-28 00:36:47
2	Screenshot 2026-03-28 003147.png	East Repair Inc.	154.06	Approved	user	2026-03-28 00:33:24
3	Screenshot 2026-03-28 002448.png	testing1	180	Approved	user	2026-03-28 00:25:07
5	Screenshot 2026-03-28 001330.png	testing1	150	Approved	user	2026-03-28 00:13:47
6	ACTIVITY.png		0	Approved	user	2026-03-27 23:44:43
7	Screenshot 2026-03-25 233522.png	testing1	140	Approved	user	2026-03-25 23:35:41
8	Screenshot 2026-03-25 231947.png	testing1	130	Approved	user	2026-03-25 23:20:20
9	Screenshot 2026-03-25 231759.png	testing1	120	Approved	user	2026-03-25 23:18:12
11	Screenshot 2026-03-25 230640.png	testing1	110	Approved	user	2026-03-25 23:06:58
12	Screenshot 2026-03-25 230429.png	testing1	110	Approved	user	2026-03-25 23:04:46

Deploy

Security Audit Log

	username	action	timestamp
0	user	Logged In	2026-03-27 23:51:08
1	user	Logged In	2026-03-27 23:44:29
2	user	Logged In	2026-03-25 23:34:54
3	user	Logged In	2026-03-25 23:20:06
4	user	Logged In	2026-03-25 23:19:12
5	user	Logged In	2026-03-25 23:16:59
6	user	Logged In	2026-03-25

admin

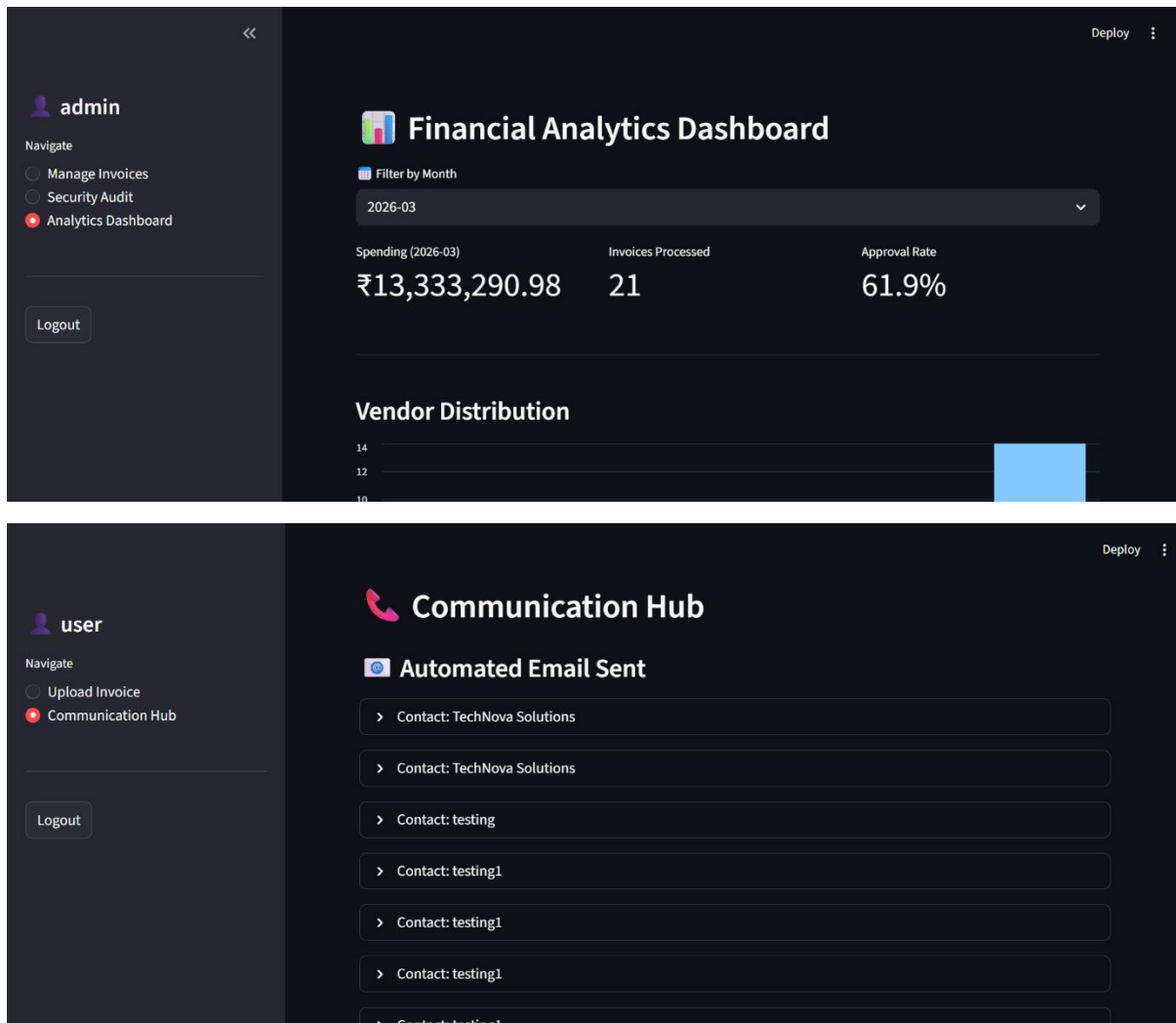
Navigate

Manage Invoices

Security Audit

Analytics Dashboard

Logout



Chapter 4: Coding

4.1 Algorithm

In this section, we define the step-by-step logical sequence that our system follows to process an invoice.

Algorithm 1: The Intelligent Invoice Processing Pipeline This algorithm dictates what happens the moment an Operator uploads a file. It acts as the gatekeeper, the reader, and the auditor all at once.

- **Step 1 [Initialization]:** Receive the uploaded file (PDF or Image) .
- **Step 2 [Cryptographic Hashing]:** * Read the file bytes.
 - Generate a 32-character MD5 hash string representing the file's unique digital fingerprint.
- **Step 3 [Duplicate Verification]:** * Query the `invoices` table in the MySQL database: `SELECT * WHERE file_hash = current_hash.`
 - *If* a match is found: Reject the file, display a "Duplicate Document" error to the Operator, and abort the process.
 - *If* no match is found: Proceed to Step 4.
- **Step 4 [AI Data Extraction]:** * Encode the file and securely transmit it to the Google Gemini Vision API via a structured prompt.
 - Receive the JSON response containing: `Vendor_Name`, `Email`, `Subtotal`, `Tax`, `Total`, and `Item_List`.
- **Step 5 [Mathematical & Policy Audit]:**
 - Calculate `Expected_Total = Subtotal + Tax.`
 - *If* `Expected_Total != Total`: Set `Error_Flag = "Math Mismatch"`.
 - *If* a keyword match is found: Set `Policy_Flag = "Policy Violation"`.
- **Step 6 [Database Persistence]:** * Determine final `Status` ("Approved" or "Flagged" based on Step 5).
 - Insert the extracted data, the generated hash, and the final status into the `invoices` database table.
 - Insert a record into the `audit_logs` table linking the action to the currently logged-in User ID.
- **Step 7 [Termination]:** Display the extracted data and final status on the Operator's dashboard.

Algorithm 2: Automated Dispute Resolution (Email Agent) This algorithm executes when an Operator clicks the "Draft & Send Dispute Email" button for a flagged invoice.

- **Step 1 [Fetch Context]:** Retrieve the flagged invoice details from the database (Vendor Name, Vendor Email, Error Reason).
- **Step 2 [Validation]:** Check if the `Vendor_Email` is valid and not null. If null, prompt the Operator for manual intervention.

- **Step 3 [Drafting]:** Dynamically generate an email subject and body using the fetched context (e.g., *"Dispute Regarding Invoice from [Vendor Name] - Math Error"*).
- **Step 4 [Transmission]:** Connect to the SMTP server (e.g., Gmail) using TLS encryption and authenticate using secure environment variables.
- **Step 5 [Execution & Logging]:** * Send the email payload.
 - *If successful:* Update the invoice status in the database to "Flagged (Email Sent)" and log the action in `audit_logs`.
 - *If failed (e.g., network error):* Catch the exception, alert the Operator, and update the status to "Flagged (Email Failed)".

4.2 Code Snippet

```
import mysql.connector
import os
from dotenv import load_dotenv

# Load secrets from .env file
load_dotenv()

def get_db_connection():
    """Establishes and returns a connection to the MySQL database."""
    try:
        conn = mysql.connector.connect(
            host=os.getenv("DB_HOST"),
            user=os.getenv("DB_USER"),
            password=os.getenv("DB_PASSWORD"),
            database=os.getenv("DB_NAME")
        )
```

```

1  import streamlit as st
2  import pandas as pd
3  import hashlib
4  import io
5  from currency_converter import CurrencyConverter
6  from database import get_db_connection
7  from auth import authenticate_user
8  from ai_engine import extract_invoice_data
9  from email_service import send_dispute_email
10
11 # Initialize Dynamic Converter
12 c = CurrencyConverter()
13
14 # --- PAGE CONFIG ---
15 st.set_page_config(page_title="FinOps AI", page_icon="🛡️", layout="wide")
16
17 # --- DYNAMIC MULTI-CURRENCY CONVERSION LOGIC ---
18
19

```

```

import google.generativeai as genai
import os
import json
from dotenv import load_dotenv

load_dotenv()
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

def extract_invoice_data(file_bytes, mime_type):
    try:
        model = genai.GenerativeModel('gemini-2.5-flash')
        prompt = """
        Analyze this document.
        Step 1: Determine if this is actually a financial receipt or not.
        Step 2: If it IS NOT a receipt (e.g., a person's photo, a random image), return exactly this JSON:
        {
            "Is Receipt": false,
            "Reason": "Not a receipt"
        }
        Step 3: If it IS a receipt, return exactly this JSON:
        {
            "Is Receipt": true,
            "Amount": "100.00",
            "Date": "2024-10-27",
            "Merchant": "ABC Company"
        }
        """

```

Chapter 5: Testing

5.1 Test Strategy

Testing a system that involves both Artificial Intelligence and strict financial rules requires a hybrid approach. Our testing strategy was divided into two main phases to ensure both the logic and the user experience were flawless:

5.2 Unit Test Plan:

Unit testing focuses on verifying the smallest parts of the application independently. For this project, we isolated the critical Python functions to ensure they worked perfectly before integrating them into the web dashboard.

- **Hash Generation Module:** We passed the exact same PDF file through the `generate_file_hash()` function 10 times to ensure it generated the exact same 32-character string every time. Then, we changed a single letter in the PDF and verified that a completely new hash was generated.
- **Math Audit Module:** We fed the `validate_invoice_data()` function simulated JSON data where Subtotal + Tax did not equal the Grand Total (e.g., $100 + 10 = 120$). We verified that the function successfully returned the "Math Error" flag and the exact error string.
- **SMTP Email Agent:** We triggered the email function with a temporary test email address to ensure the connection to Gmail's server was secure and the formatting of the dispute email was professional and readable.

5.3 Acceptance Test Plan

Also known as User Acceptance Testing (UAT), this phase ensured that the system actually solved the business problem for our target users: the Operator and the Admin.

- **Operator Acceptance:** Can an intern with zero training successfully upload an invoice, understand the visual output and trigger a dispute email without asking for help?
- **Admin Acceptance:** Can the Finance Manager log in, view the analytics charts without seeing any operational upload buttons, and successfully download the monthly report into an Excel file?

- **Security Acceptance:** Does the system successfully block the Operator from navigating to the Admin's audit_logs page?

5.4 Test Case / Test Script

Here is a snapshot of our formal test scripts. We created structured test cases to systematically check the core functional requirements.

Test Case ID	Feature Tested	Action Performed (Input)	Expected Result	Actual Result	Status
TC-01	Duplicate Prevention	Upload bill_01.pdf. Then immediately upload bill_01.pdf again.	System should block the second upload and show "Duplicate File" error.	System blocked the upload and displayed the error.	☒ PASS
TC-02	AI Extraction Accuracy	Upload a clear PDF invoice from "Dell".	System should correctly extract "Dell" and the Grand Total.	Vendor: Dell, Total: \$500 extracted correctly.	☒ PASS
TC-03	Math Validation	Upload an invoice where Subtotal is \$100, Tax is \$10, but Total says \$150.	System should flag the invoice with a "Math Error".	System flagged invoice. Status turned Red (●).	☒ PASS
TC-04	Role-Based Access	Log in using Operator credentials and attempt to view the Analytics page.	System should hide the Analytics module from the sidebar.	Analytics module was completely hidden from the Operator.	☒ PASS

Chapter 6: Limitations of the Proposed System

Every software system has its boundaries, and acknowledging them is a crucial part of professional software engineering. While our AI-Powered Financial Operations Automation System drastically reduces manual workload, it currently has a few limitations based on its scope and technology stack:

1. **Dependency on External APIs & Internet Connectivity:** Because the core extraction engine is powered by the Google Gemini Vision API, the system is fundamentally cloud-dependent. If the host organization experiences an internet outage, or if Google's API undergoes temporary maintenance, the invoice processing pipeline will halt. It cannot process documents entirely offline.
2. **Highly Distorted or Illegible Documents:** While the multimodal AI is incredibly smart, it still relies on visual clarity. If an operator uploads a severely crumpled, faded, or heavily stained physical receipt, or if a handwritten invoice uses highly illegible cursive handwriting, the AI's confidence score drops. In these edge cases, it might misread a digit or fail to extract the vendor name, requiring manual intervention.
3. **Lack of Direct Bank/Payment Integration:** Currently, the system acts as a smart "Gatekeeper and Auditor." It successfully approves an invoice and logs it into the database as "Ready for Payment." However, it does not actually move the money. The finance team still needs to take the exported Excel ledger and manually initiate the NEFT, RTGS, or Swift transfers through their corporate banking portal.
4. **Language Constraints:** For this initial prototype, the system's policy-checking logic and automated dispute emails are hardcoded in English. While the AI can read regional languages, our backend arrays (like checking for the word "alcohol" or "entertainment") are currently limited to English vocabularies.

Chapter 7: Proposed Enhancements

Software is never truly "finished." If this project were to receive seed funding or be adopted by a larger enterprise, here is the roadmap for how we would evolve it from an internal tool into a massive commercial SaaS (Software as a Service) product.

1. **Seamless ERP and Accounting Software Integration:** Instead of forcing the Admin to download an Excel sheet at the end of the month, the next logical step is to build API bridges to major accounting platforms like Tally Prime, QuickBooks, or SAP. Once an invoice is marked as "Approved" in our system, it would automatically populate in the company's main ledger without any human clicks.
2. **Direct Payment Gateway Integration:** To solve the limitation mentioned in Chapter 6, we would integrate the system with banking APIs (like RazorpayX or Stripe Connect). Once the AI audits the math and the Admin gives a final click of approval, the system would automatically disburse the funds directly to the vendor's bank account.
3. **Predictive Analytics & Budget Forecasting (Machine Learning):** Currently, our analytics dashboard looks at the *past* (e.g., "How much did we spend last month?"). A future enhancement would involve training a local Machine Learning model (using Scikit-Learn or TensorFlow) on our historical MySQL data to predict the *future*. The dashboard could warn the Admin: *"Based on current upload trends, you will exceed your IT hardware budget by the 20th of this month."*
4. **Dedicated Mobile Companion App:** For employees who travel often (like sales teams), we would develop a lightweight Android/iOS application. Instead of waiting to get back to the office to scan a hotel bill, the employee could just snap a picture on their phone. The app would instantly compress the image, hash it, and send it to our Python backend for immediate processing.

Chapter 8: Conclusion

The traditional Accounts Payable workflow—reliant on manual data entry and human auditing—is no longer sustainable for modern enterprises. It is slow, highly susceptible to expensive errors, and leaves companies vulnerable to duplicate payment fraud and policy violations.

This project, the **AI-Powered Financial Operations Automation System**, successfully resolves these critical inefficiencies. By integrating Google Gemini's Multimodal AI with cryptographic MD5 hashing, the system intelligently extracts invoice data while structurally eliminating duplicate payments. The automated Validation Engine mathematically audits transactions and enforces corporate policies instantly. Crucially, the Automated Dispute Agent removes the human bottleneck by independently drafting and dispatching vendor emails when anomalies are detected.

Secured by Role-Based Access Control and immutable audit logs, this system proves that Artificial Intelligence does not replace finance teams, but empowers them—automating repetitive chores so professionals can focus on strategic, high-value financial growth.

Chapter 9: Bibliography

References:

- [1] Python Software Foundation, "Python 3.9 Reference Documentation," *docs.python.org*. [Online]. Available: <https://docs.python.org/3/> (Reference for the core backend programming language, *smtp*lib, and *hashlib*).
- [2] Streamlit Inc., "Streamlit Documentation - Build and Share Data Apps," *docs.streamlit.io*. [Online]. Available: <https://docs.streamlit.io/> (Reference for the frontend UI framework and state management).
- [3] Google DeepMind, "Gemini API Documentation: Multimodal AI and Vision Capabilities," *ai.google.dev*. [Online]. Available: <https://ai.google.dev/docs> (Reference for the AI extraction engine used in the project).

- [4] Oracle Corporation, "MySQL 8.0 Reference Manual," *dev.mysql.com*. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/> (*Reference for the relational database schema and 3NF normalization*).
- [5] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY: McGraw-Hill Education, 2014. (*Standard academic reference to justify your System Requirements, ERD, and SDLC methodology*).